
Shockwave Graph Neural Networks

Urszula Julia Komorowska

Department of Computer Science and Technology
University of Cambridge

Supervisors: **Cristian Bodnar** and **Alexander Norcliffe**

Abstract

The new interesting perspective on Graph Neural Networks analyses them as a dynamical systems governed by some Partial Differential Equations. Prominent examples of this approach are works on the graph diffusion or attempts to capture graph information flow with the non-linear hyperbolic equations. One of the issues those novel architectures address is the over-smoothing phenomenon. This project aims to explore the potential of Burgers' equation for modeling the message flow in the graph. It proposes a model based on this equation which has a property of a pure Laplace diffusion and attentional diffusion. The model is evaluated on the node classification task in a number of different scenarios. Firstly, the general dependency of performance on the viscosity and time step is investigated. Then the experiments are repeated when viscosity is a learnable parameter, lastly when one more learnable parameter is introduced as a weight of a shock term. This report does not find benefits from making viscosity and shock weight learnable parameters. The step size and initialisation of viscosity are found to be more important for the final performance.

1 Introduction

Graph Neural Networks (GNNs) have proved to be very useful in a wide range of problems from social network analysis (1) to physical sciences (2), (3). Recently it has been shown that links between physical sciences and GNNs are not merely limited to the application potential, but concepts from physics can be transferred to the domain of machine learning and lead to improved graph representations. The field of Partial Differential Equations (PDEs) offers an interesting perspective on the analysis of GNNs.

Although the analogy between the two is not immediately obvious, the first step towards interpreting GNNs as PDEs was reformulating the Convolutional Neural Network such that each convolution layer is considered a time step of a discretized PDE (4). The convolutional layer can be generalised to graphs (5) and the analogy to PDEs extends to this domain. Eliasof et al. (6) show GCNs can be viewed as discretizations of PDEs on manifolds using the examples of non-linear diffusion and non-linear hyperbolic equations. The graph diffusion is analysed in depth by Chamberlain et al. in the work introducing the GRAND (Graph Neural Diffusion) framework (7). They study different numerical schemes for solving the graph diffusion equation and experimentally show this new class of GNNs can result in superior performance on popular benchmark datasets. This is achieved by the appropriate choice of the discretisation and the numerical schemes which alleviate over-smoothing. The insights from the GRAND framework can be leveraged to study more complex graph PDEs with a perspective of finding architectures more robust to over-smoothing. Another candidate equation for modelling dynamics of the graph can again come from the field of the physical sciences, for example the Burgers' equation that connects diffusion with the shock formation.

Sections of this report are concerned with the following topics:

- Section 2 recalls the theory of the diffusion equation on the graph. It also explains the theory behind the Burgers' equation and proposes appropriate spatial discretisation scheme, which is variation of the already existing GRAND architecture.
- Section 3 explains how the experiments were performed and Section 4 shows the result. Subsection 4.1 analyses the performance when the viscosity is a constant and aims to find a general influence of the time step size and viscosity on the performance.
- Subsection 4.2 checks how the model is affected if the viscosity is a learnable parameter. Then, in the Subsection 4.3 the update function is modified to have the shock term multiplied by some c , which also is a learnable parameter. It is checked if this additional control of size of shock term compared to Laplace diffusion term is beneficial.
- Finally, Section 5 is a discussion of the results and gives suggestion for the further research directions.

2 Theoretical background and motivation

2.1 Attentional diffusion

This Section outlines the mathematics of the diffusion equation and explains how it is represented on the graph when both space and time discretisations are necessary. This is required since graphs are not a continuous Euclidean space, and graph features are modified iteratively with the action of the network layers.

In general terms, the diffusion equation describes the behavior of many particles resulting from random motion and collision. The overall effect is the flow of the substance from the region of high concentration to low concentration. Let $u(x, t)$ be a scalar valued function on a space-time domain $\Omega \times [0, \infty)$. The diffusion dynamics are expressed by:

$$\frac{\partial u(x, t)}{\partial t} = \nabla \cdot (D(x, u(x, t)) \nabla u(x, t)) \quad (1)$$

where $D(x, u(x, t))$ is the diffusion coefficient. Most generally D is a function of space and $u(x, t)$, but often it is set to be a constant. With the appropriate change of the function and the operator, this equation can be reformulated into the graph discretised version.

Given the undirected graph $G = (V, \mathcal{E})$ with $|V| = n$ nodes $|\mathcal{E}| = e$ edges, define $\mathbf{x}$ to be an n -dimensional vector field of node features. The node and edge features are assumed to be scalar valued in analogy to the scalar valued $u(x, t)$, but the reasoning also applies to the vector valued features. Also define $\mathcal{X}$ to be an e -dimensional vector of edge features (weights). The graph diffusion equation as defined in (7) takes the form of:

$$\frac{\partial \mathbf{x}(t)}{\partial t} = \nabla \cdot (\mathbf{D}(\mathbf{x}(t), t) \nabla \mathbf{x}(t)) \quad (2)$$

The divergence and gradient operators no longer act in the Euclidean space and must be redefined on a graph domain. The gradient $(\nabla \mathbf{x})_{ij} = x_j - x_i$ assigns to each edge $(i, j) \in \mathcal{E}$ the difference in node features. The divergence operator assigns each node a sum of features of all edges attached to it, i.e.

$$(\nabla \cdot \mathcal{X})_i = \sum_{j: (i, j) \in \mathcal{E}} \mathcal{X}_{ij} \quad (3)$$

Chamberlain et al. propose $\mathbf{D}(\mathbf{x}(t), t)$ to be the diagonal matrix filled in with the attention coefficients for each edge: $\mathbf{D} = \text{diag}(a(\mathbf{x}_i(t), \mathbf{x}_j(t)))$. The explicit dependence on time in $\mathbf{D}$ is removed for simplicity. The interpretation is that for each node i the node feature 'diffuses' along the edges weighted with attention coefficients, and flow along all edges is aggregated for each node. After substituting the above definitions to 3, operations can be represented in the matrix form:

$$\frac{\partial \mathbf{x}(t)}{\partial t} = (\mathbf{A}(\mathbf{x}(t)) - \mathbf{I})\mathbf{x}(t) \quad (4)$$

In this attentional diffusion, $\mathbf{A}(\mathbf{x}(t))$ is not constant and changes in time. The final piece of work requires finding the good time discretisation scheme. Many are possible and the simplest one is the forward Euler scheme:

$$\mathbf{x}^{t+\Delta t} = \mathbf{x}^t + \Delta t(\mathbf{A}(\mathbf{x}(t)) - \mathbf{I})\mathbf{x}(t) \quad (5)$$

stable for the time step size $\Delta t < 1$, reported in (7).

2.2 Burgers' equation

In physics, the Burgers' equation describes the flow of the fluid with velocity $u(x, t)$ and viscosity ν through a one dimensional pipe at time t as:

$$\frac{\partial u}{\partial t} + c(u) \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2} \quad (6)$$

where $c(u)$ is some function of u . The time derivative of u can be represented as a sum of two terms: diffusion (weighted by the viscosity) and the term that under specific conditions can lead to the development of shockwaves.

This again can be represented on the graph $G = (V, \mathcal{E})$ if the operators are reformulated and the velocity field $u(x, t)$ is replaced with the feature matrix $\mathbf{X}$. Action of the Laplace operator ∇^2 is equivalent to the multiplication by the minus combinatorial Laplacian $\mathbf{L}$ (minus signifies diffusion out of the node). This is a standard method of representing pure diffusion on the general graph (6). Moreover, the entire expression $c(u)\nabla$ is equated to $(\mathbf{I} - \mathbf{A})$, same as the attentional diffusion mechanism but with reverse sign. The reason for that is that when $(\mathbf{I} - \mathbf{A})$ is taken to the right side of the equation, $\partial_t \mathbf{X}$ is more negative if features 'diffuse' out of the node, in other words when there is a positive concentration gradient. This is equivalent to the more negative $\partial_t u$ if $c(u)\partial_x u > 0$, meaning there is a positive gradient if $c(u) > 0$. This is the physical case of $c(u) = u$ while modeling a real fluid flow.

The Equation 6 now reads

$$\frac{\partial \mathbf{X}}{\partial t} = \nu \mathbf{L} \mathbf{X} + (\mathbf{A} - \mathbf{I}) \mathbf{X}$$

Similarly like in 2.1 the equation must be time discretised. Again, this report assumes the forward Euler scheme:

$$\mathbf{X}^{t+\Delta t} = [\Delta t \cdot (\nu \mathbf{L} + (\mathbf{A} - \mathbf{I})) + \mathbf{I}] \mathbf{X}^t$$

The potential benefits of using the Burgers' equation will be outlined with the help of the Dirichlet energy. The model based on the Burgers' equation will be referred to as the *Burgers' model*.

In the simplest terms, the Dirichlet energy quantifies the 'smoothness' of a function (8). A function that greatly varies within its domain will have high Dirichlet energy. The neural network, which in fact is described by some function, will lose its expressiveness if its Dirichlet energy is too low.

Given the undirected graph $G = (V, \mathcal{E})$ let $\bar{\mathbf{A}}$ and $\mathbf{D}$ be its adjacency and degree matrices respectively. Let $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2} \bar{\mathbf{A}} \mathbf{D}^{1/2}$ be the symmetric normalised Laplacian and $\mathbf{X}$ be a feature matrix. The Dirichlet energy of $\mathbf{X}$ is defined as:

$$E(\mathbf{X}) = \text{tr}(\mathbf{X}^T \mathbf{L} \mathbf{X}) = \frac{1}{2} \sum_{(i,j) \in \mathcal{E}} w_{ij} \left\| \frac{\mathbf{X}_i}{\sqrt{d_i}} - \frac{\mathbf{X}_j}{\sqrt{d_j}} \right\|_2^2 \quad (7)$$

where w_{ij} are edge weights, $\mathbf{X}_i$ is the i^{th} row of $\mathbf{X}$, d_i is the degree of node i . The energy decreases for models based on diffusion, but in a different manner for the Burgers' model and pure diffusion.

Let $\mathbf{M} = \Delta t \cdot (\nu \mathbf{L} + (\mathbf{A} - \mathbf{I})) + \mathbf{I}$. Define $\mathbf{V}_i$ to be an n dimensional row vector filled in with zeros, except with 1 at the i^{th} position. The multiplication $\mathbf{V}_i \mathbf{X}$ gives $\mathbf{X}_i$. Then the suprema of the Dirichlet energy at different time steps are:

$$E(\mathbf{X}) = \frac{1}{2} \sum_{(i,j) \in \mathcal{E}} w_{ij} \left\| \frac{\mathbf{V}_i \mathbf{X}}{\sqrt{d_i}} - \frac{\mathbf{V}_j \mathbf{X}}{\sqrt{d_j}} \right\|_2^2 \leq \frac{1}{2} \sum_{(i,j) \in \mathcal{E}} w_{ij} \left\| \frac{\mathbf{V}_i}{\sqrt{d_i}} - \frac{\mathbf{V}_j}{\sqrt{d_j}} \right\|_2^2 \|\mathbf{X}\|_2^2 \quad (8)$$

$$\begin{aligned} E(\mathbf{M}\mathbf{X}) &= \frac{1}{2} \sum_{(i,j) \in \mathcal{E}} w_{ij} \left\| \frac{\mathbf{M}\mathbf{V}_i \mathbf{X}}{\sqrt{d_i}} - \frac{\mathbf{M}\mathbf{V}_j \mathbf{X}}{\sqrt{d_j}} \right\|_2^2 \leq \|\mathbf{M}\|_2^2 \frac{1}{2} \sum_{(i,j) \in \mathcal{E}} w_{ij} \left\| \frac{\mathbf{V}_i \mathbf{X}}{\sqrt{d_i}} - \frac{\mathbf{V}_j \mathbf{X}}{\sqrt{d_j}} \right\|_2^2 \leq \\ &\leq \|\mathbf{M}\|_2^2 \frac{1}{2} \sum_{(i,j) \in \mathcal{E}} w_{ij} \left\| \frac{\mathbf{V}_i}{\sqrt{d_i}} - \frac{\mathbf{V}_j}{\sqrt{d_j}} \right\|_2^2 \|\mathbf{X}\|_2^2 \end{aligned}$$

$$E(\mathbf{M}\mathbf{X}) \leq \|\mathbf{M}\|_2^2 E(\mathbf{X})$$

	Texas	Cornell
# Nodes	183	183
# Edges	309	295
# Classes	5	5

Table 1: Datasets statistics.

$$\sup(E(\mathbf{MX})) = \|\mathbf{M}\|_2^2 \sup(E(\mathbf{X}))$$

$$\sup(E(\mathbf{MX})) = \sigma(\mathbf{M}) \sup(E(\mathbf{X}))$$

where $\sigma(\mathbf{M})$ is the largest eigenvalue of $\mathbf{M}$.

Knowing that the eigenvalues of the diffusion Laplacian are in a range $(-2, 0)$ and $\mathbf{A}$ is the stochastic matrix with eigenvalues in range $(-1, 1)$, the upper bound on the largest eigenvalue of $\mathbf{M}$ is 1. $\mathbf{A}$ is the attention matrix whose values depend on the current features, so that $\mathbf{M}$ is not the same for all layers. Energy decrease from layer to layer varies and depends on viscosity, time step and attention weights, such that network can control the drop in the Dirichlet energy.

What the equations above are showing is that this is a supremum that changes in a way predicted by $\sigma(\mathbf{M})$, not necessarily the exact value of the energy at the moment. It however still shows that the size of the changes remains under control of the network.

In a case of the pure Laplacian diffusion the energy also decreases. Let $\mathbf{M} = \Delta t \cdot (\nu \mathbf{L} + \mathbf{I})$. The change of the Dirichlet energy in a single update step is:

$$\mathbf{X}^{t+1} = \mathbf{MX}^t \tag{9}$$

for $\mathbf{M}$ whose largest eigenvalue is also upper bounded by 1. The only learnable parameter is the viscosity, staying the same for all layers.

With the time step defined in the way above, the model puts the shock part on top of the pure Laplace diffusion. Moreover, it should be able to decrease the amount of pure diffusion if necessary. Like in any other iterative scheme for solving PDEs, the change in the function values is assumed to be small in comparison with the value of the function at the point. Hence too large a viscosity will cause the change in features to be too big as well. This report will analyse if it is possible to achieve a balance between two parts of the update rule that will increase expressivity without fatal over-smoothing.

3 Methods

The experiments were performed on two datasets, whose statistics are presented in the Table 1. Texas and Cornell datasets are webpage datasets collected from computer science departments. Nodes are web pages, whose features are their bag-of-words representation. Edges are hyperlinks between pages. Graphs were assumed to be undirected and the self loops were removed.

In the Burgers' model, one time step forward corresponds to one pass through the layer. The PyTorch implementation of the model uses the normalised Laplacian $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2} \bar{\mathbf{A}} \mathbf{D}^{1/2}$ for the pure diffusion part of the update (without the normalisation, features would blow up). The shock part of the update was the attention matrix implemented using the PyTorch Geometric GATConv layer with one head, no self loops and hidden dimension 64. The learning rate was set to 0.01. At each experiment, model was trained independently when number of layers was varied.

The GCN baseline was implemented using PyTorch Geometric GCNConv layer with hidden dimension 64. The best results for GCN were obtained when the learning rate on the Texas dataset was 0.1 and on Cornell was also 0.1. They are close to the learning rate obtained by the hyperparameter search on those two datasets (9). The weight decay was set to 0.0005, dropout was set to 0.5. The Texas and Cornell datasets come with 10 default splits, such that the final test accuracy is the average on those. The loss function on the node classification task was the negative log likelihood loss, the models were trained with the early stopping of patience 20.

4 Results

4.1 Non-trainable viscosity

The first set of experiments were performed with the viscosity set to a constant value. This Section attempts to figure out if there is any straightforward relationship between ν , Δt and accuracy. At the stage of the model design it is hard to determine what the right Δt should be and what ν it ideally corresponds to. In order to have a better understanding of the interplay of those parameters, the general search over them is performed. Figures 1 and 2 show how accuracy changes for different ν , Δt and number of layers.

This experiment, as well as the following ones, does not aim to find a model superior to currently available frameworks which would require the extensive hyperparameter search and evaluation on diverse datasets. It is rather trying to find the simplest way to improve the model by finding the balance between shock and diffusion. Once the way to achieve this balance is known, it might be a potential starting point for designing the best version of the model possible.

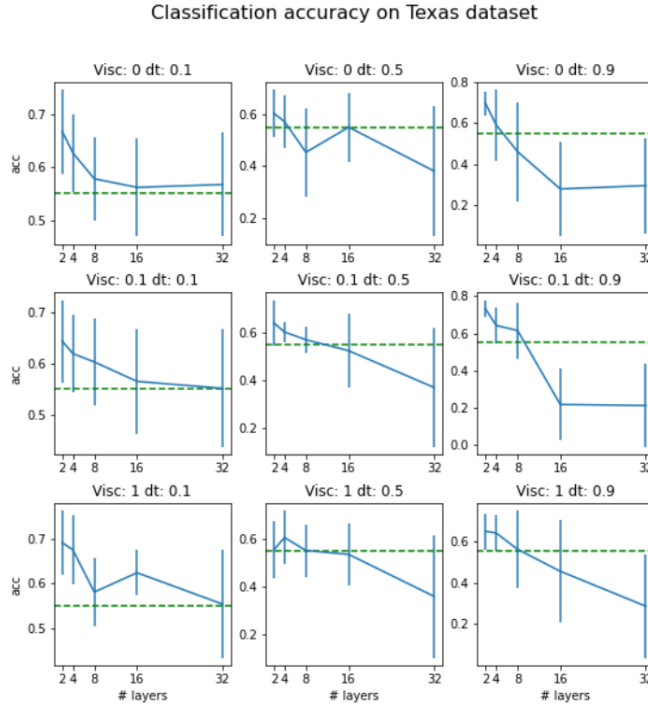


Figure 1: First set of experiments on Texas. Blue line shows the accuracy of the Burgers' model on the node classification task. Dashed green line shows the GCN accuracy. The best accuracy reported by the fine-tuned GRAND is about 75% (10).

Similar behaviour is visible on all the plots of Figures 1 and 2. For small number of layers, the accuracy can outperform simple GCN. For example, on a Texas dataset it can go as high as 73.2% for $\nu = 0.1$, $\Delta t = 0.9$. For $\Delta t = 0.5$ and $\Delta t = 0.9$ when the over-smoothing hits the accuracy drops significantly below the baseline level to as low as 20% (random guessing). Over-smoothing manifests similarly for other ν , Δt , but more often even with that the Burger's model is comparable with GCN. On the Cornell dataset the time step value seems to have bigger influence on the accuracy. As opposed to the results on the Texas dataset, the Burgers' model will not outperform GCN even for small number of layers when $\Delta t = 0.9$.

There is some optimal set of parameters ν , Δt defining how well the Burgers' model can perform for smaller number of layers and what is the maximum accuracy it can get. This relationship is not clear: for example on the Texas dataset, the best accuracy is achieved for no viscosity or its intermediate value of 0.1 with the biggest time step, but for other pairs ν , Δt the best accuracies remain in the range 61-70%. From the Figures 1, 2 it is not possible to infer what the best set of parameters might

Classification accuracy on Cornell dataset

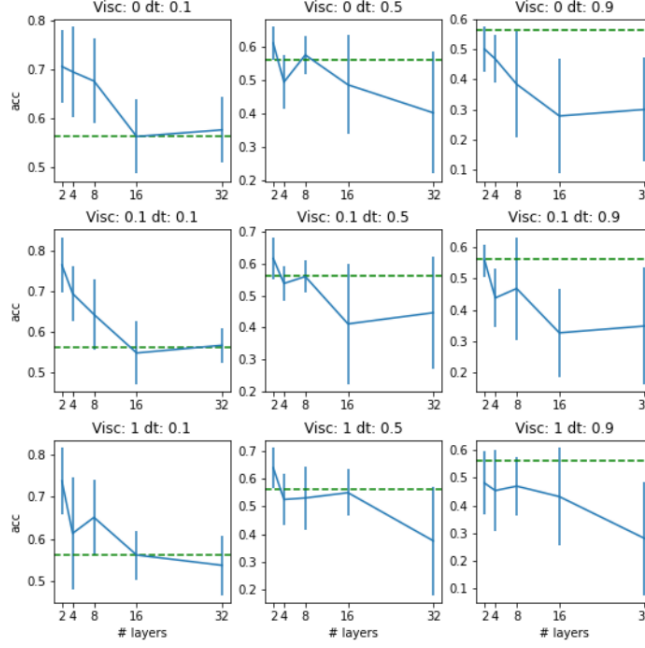


Figure 2: First set of experiments on Cornell. Blue line shows the accuracy on the node classification task of the Burgers’ model. Dashed green line shows the GCN accuracy. The best accuracy reported by the fine-tuned GRAND is about 82% (10).

look like, but it shows that for a given time step there is a best viscosity value to be found. Moreover, carefully chosen ν , Δt might results in much improved accuracy without fatal over-smoothing for number of layers bigger than 8.

As for any ordinary differential equation, $\partial_t \mathbf{X} \cdot \Delta t$ must be small compared to $\mathbf{X}$. If this condition is not met, which might be caused by a time step or viscosity which is too large, Burgers’ model might develop instabilities. That could be the potential explanation for the big error bars for some pairs $\nu, \Delta t$ or bigger number of layers.

The next set of experiments aims to find the optimal viscosity for the given Δt .

4.2 Viscosity as a trainable parameter

The experiments in Figures 3 and 4 check the hypothesis that with a carefully chosen ν the accuracy should improve. The model was initialised for 8 different ν values, one of them being approximately zero. Viscosity should always be kept positive, otherwise the diffusion is interpreted as flow into the node even if the neighbours’ features are lower. To ensure that, the additional variable of *raw* viscosity is included, such that $\nu = \exp(\nu_{raw})$. The ν_{raw} was drawn from the standard normal distribution. For the one experiment when viscosity is said to be initialised to approximately zero, the initial ν_{raw} was explicitly set to -10.

4.2.1 Texas dataset

For $\Delta t = 0.1$, addition of the learnable viscosity can offer some improvement in accuracy. The best performance for number of layers 2 and 4 is achieved when there is very little viscosity. This goes down for the intermediate values of ν and after that there is no significant change when ν_{raw} is closer to 1. In the case when optimisation of ν was started from approximately zero, the model does not converge to a value very different from that: for example, in one instance of the model the final ν_{raw} was about -10.14.

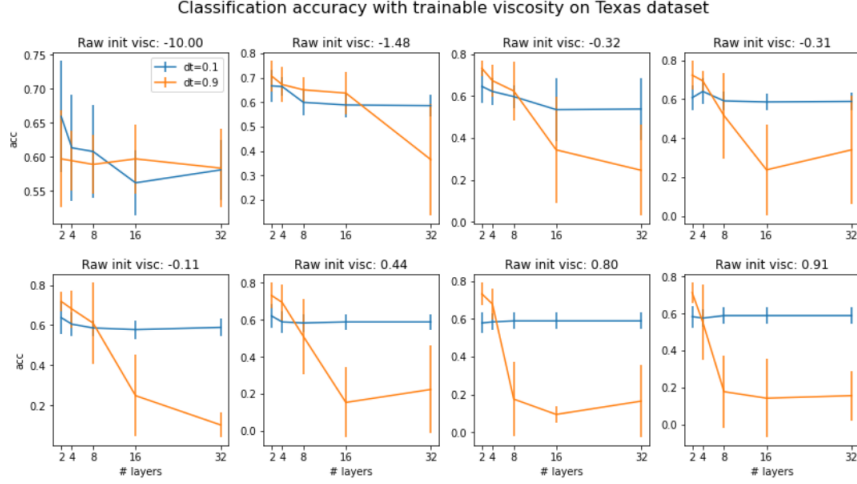


Figure 3: Node classification accuracy when viscosity is a trainable parameter, Texas dataset.

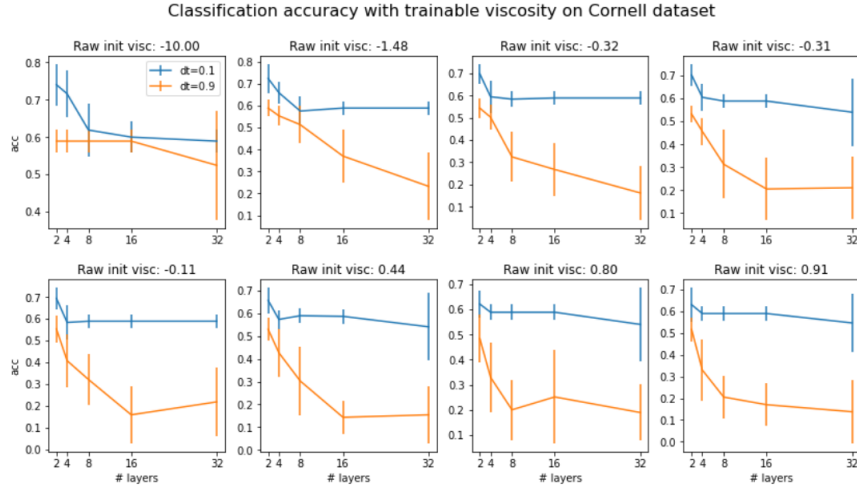


Figure 4: Node classification accuracy when viscosity is a trainable parameter, Cornell dataset.

Next analysed scenario was $\Delta t = 0.9$. Again, adding some small amount of viscosity is beneficial for accuracy for a small number of layers. However, the accuracy drops to not better than random guessing for 16 and 32 layers if too much viscosity is added, while model with no viscosity stays at the level of above 50% accuracy.

4.2.2 Cornell dataset

Regardless of the time step chosen, introducing more viscosity only reduces the accuracy. The over-smoothing manifests similarly as in the case when the viscosity is not trainable. The relationship that bigger viscosity will cause more dramatic over-smoothing still persists.

4.2.3 General comments

There is no clear rule that would determine when the viscosity should be added. For too large Δt adding ν can have a detrimental effect, but what “too big Δt ” means on a particular dataset is hard to determine. Moreover, adding this one more parameter to optimise increases complexity of the training, increasing the risk of optimiser getting stuck in the local optimum.

For each pair $\nu_{raw}, \Delta t$ the model was trained again when initialised to a different number of layers, hence ν_{raw} was converging to different values. Table 2 presents examples of what is the span of

Texas dataset, $\Delta t = 0.1$			Cornell dataset, $\Delta t = 0.1$		
Initial raw visc	2 layers	32 layers	Initial raw visc	2 layers	32 layers
-1.48	-1.74	-1.42	-1.48	-1.77	-1.38
-0.32	-0.44	-0.28	-0.32	-0.55	-0.26
-0.31	-0.45	-0.27	-0.31	-0.54	-0.23
-0.11	-0.28	0.01	-0.11	-0.32	-0.03
0.44	0.22	0.47	0.44	0.20	0.51
0.80	0.56	0.85	0.80	0.45	0.77
0.91	0.67	0.93	0.91	0.66	0.86

Table 2: Examples of final values to which ν_{raw} converges on two datasets.

changes of final ν_{raw} values. For smaller number of layers, there is a trend of lowering the viscosity which disappears as the number of layers increases. Model aims to lower amount of the pure diffusion. This however changes when there are more layers (more time steps forwards) and the deviations from the initial values are smaller. A potential explanation for this might be a fact that when there are more update steps, the effects of small change in ν accumulate across the layers and the final impact on the output is large. In any case, the initial ν_{raw} is indicative of what its final value will be, hence annotations of the Figure 3 and 4 report what was initial ν_{raw} .

Ideally, for a given Δt , performance for any number of layers should improve if the appropriate ν can be found. However, in the optimisation process the best ν is unlikely to be found, and jumps in accuracy do not occur in a principled manner.

Another aspect of the above experiments is that even if adding some small amount of viscosity is beneficial for some cases, the over-smoothing still persists. This is because $\partial_t \mathbf{X} \cdot \Delta t$ is not small enough compared to $\mathbf{X}$ and the aggregated effect of many time steps causes the nodes to be indistinguishable. One way to resolve this problem is to decrease ν , but that already turned out to be difficult for the model under certain conditions. Another would be to modify the update equation to:

$$\mathbf{X}^{t+\Delta t} = [\Delta t \cdot (\nu \mathbf{L} + c(\mathbf{A} - \mathbf{I})) + \mathbf{I}] \mathbf{X}^t \quad (10)$$

where c is the additional learnable parameter. In this way, for any given Δt chosen by the experimenter, the model is able to make $\partial_t \mathbf{X}$ small enough to alleviate over-smoothing.

4.3 Adding learnable shock parameter

The experiments with the learnable shock parameter were performed for initial $\nu_{raw} = -1.48$ and $\nu_{raw} = 0.44$ for $\Delta t = 0.1$ and $\Delta t = 0.9$. That allows for the direct comparison with results from Figures 3 and 4. For each initial ν_{raw} and Δt two starting points of the training were analysed: initial $c = 0.5$ or $c = 0.9$.

4.3.1 Texas dataset

Results are presented on Figures 5 and 6. No significant improvement was found.

4.3.2 Cornell dataset

Introduction of the learnable shock coefficient does not result in the significant improvement. Similarly as when only the viscosity is learnable, for $\Delta t = 0.1$ the accuracy drops to the level of about 60% for 8 layers and more. The only case when the over-smoothing is alleviated is when $\Delta t = 0.9$ and initial $\nu_{raw} = -1.48$, then the accuracy remains at the level of about 65% for 8 layers.

4.3.3 General comments

Table 3 shows what is the trend in convergence for c and ν_{raw} . For 2 layers both of those parameters are decreased, while for 32 layers this drop is less pronounced. Most often, the model learns to decrease $\partial_t \mathbf{X}$ by lowering viscosity and shock coefficient.

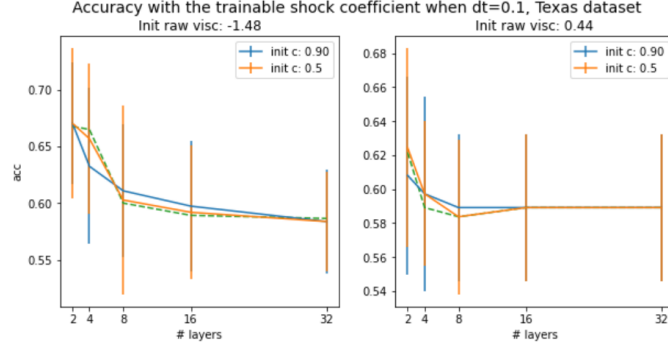


Figure 5: Accuracy when the shock coefficient is learnable for $\Delta t = 0.1$, Texas dataset. Green dashed line shows accuracies when only the viscosity was learnable and shock coefficient was constant and equal to 1.

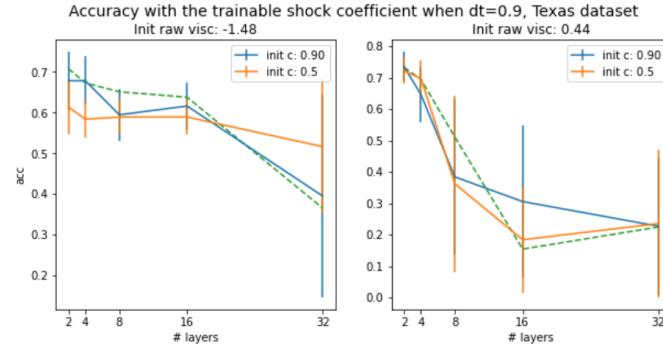


Figure 6: Accuracy when the shock coefficient is learnable for $\Delta t = 0.9$, Texas dataset. Green dashed line shows accuracies when only the viscosity was learnable and shock coefficient was constant and equal to 1.

Overall, the benefits from adding the trainable c are not systematic. There is not enough strong evidence that this will be helpful for all sets of Δt and ν .

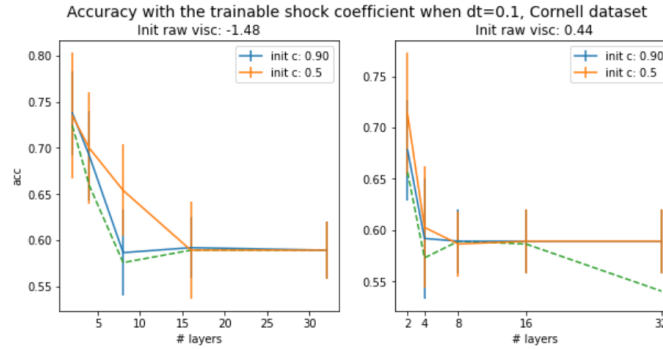


Figure 7: Accuracy when the shock coefficient is learnable for $\Delta t = 0.1$, Cornell dataset. Green dashed line shows accuracies when only the viscosity was learnable and shock coefficient was constant and equal to 1.

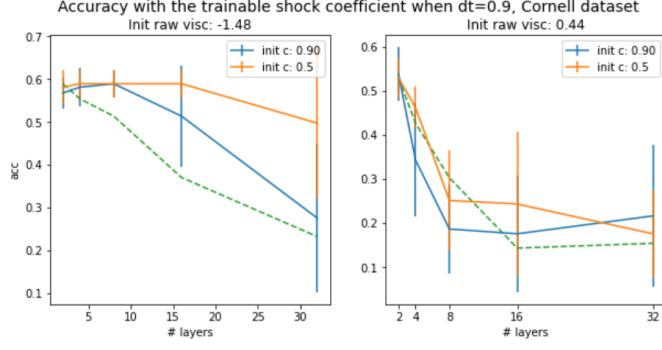


Figure 8: Accuracy when the shock coefficient is learnable for $\Delta t = 0.9$, Cornell dataset. Green dashed line shows accuracies when only the viscosity was learnable and shock coefficient was constant and equal to 1.

Cornell dataset, $\Delta t = 0.1$		2 layers		32 layers	
Initial shock coeff	Initial raw visc	shock	visc	shock	visc
0.90	-1.48	0.70	-1.69	0.83	-1.42
0.90	0.44	0.65	0.17	0.85	0.39
0.50	-1.48	0.34	-1.65	0.43	-1.39
0.50	0.44	0.24	0.18	0.57	0.52

Table 3: Examples of values to which c and ν_{raw} can converge.

5 Discussion and conclusions

Summary This report performs the investigation of the performance of the Burgers’ model when the properties of the Laplace and attentional diffusion are mixed in a hope of alleviating over-smoothing.

The Burgers’ model was evaluated on the node classification task in different scenarios. Firstly, the general search for the dependency of accuracy on time step and viscosity was performed. Then the viscosity was set to be learnable parameter and experiments were repeated for two time step sizes. Lastly, the additional parameter named the shock coefficient was introduced, such that the model had control over the contribution of both terms in the update rule.

In agreement with the general PDE theory, it is of crucial importance to have $\partial_t \mathbf{X} \cdot \Delta t$ small enough. The good choice of Δt and initial viscosity turned out to be most important factors.

Interplay of viscosity, shock coefficient and time step size For Δt too large, the Burgers’ model will suffer from over-smoothing and the most effective way to prevent that is to decrease Δt . Manipulating ν plays a second role. There were not many benefits from making ν learnable and similarly good accuracy for number of layers equal to 2 or 4 could be achieved by setting ν to be a constant small value. The Burgers’ model has a capacity to adjust ν if made trainable, possibly giving the experimenter a bigger margin of error when defining Δt , that should be however subject to further investigation. In a similar way the effects of introducing the learnable shock coefficient c should be investigated. In theory the Burgers’ model can decide what is the best ratio ν/c , but in practice it is hard to achieve since values of ν and c do not diverge much from their initial values, hence restricting the range of the ratio ν/c . The more likely case is that the model will decrease both these coefficients to account for the possibly too large value of Δt . Locally optimal values for ν/c can be found in this process, but it remains undetermined what would be the best possible ratio for the given Δt .

Further work An interesting avenue for further work would be to replace the fixed time step iterative scheme with the adaptive step size method. The example of that is a NeuralODE framework by Chen et al. (11), also transferred into the domain of graph representation learning by Poli et al. (12). Similarly like in this report, they represent the change in features (or more generally, values on the hidden layers) as the time derivative: $\partial_t \mathbf{h}(t) = f(\mathbf{h}(t), t, \theta)$ where the right hand side is a

function of features, time and network parameters θ . In their scheme, the integration is performed from T_{start} to T_{final} as if it was the ordinary differential equation for $\mathbf{h}$. Updates are done using the well established SciPy solvers such as `dopri8` (13) with the adaptive time step size. At each step, the size of Δt is adjusted such that the estimated integration errors stays below some pre-defined threshold. The features are propagated until T_{final} and then backpropagation takes place. In this way, the user does not need to specify Δt explicitly. The difficulty might be in finding the appropriate integration time T_{final} . This is conceptually equivalent to finding the maximum number of layers for the Euler scheme before the over-smoothing decreases the performance.

References

- [1] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, “Graph convolutional neural networks for web-scale recommender systems,” in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ACM, 2018.
- [2] J. Shlomi, P. Battaglia, and J.-R. Vlimant, “Graph neural networks in particle physics,” *Machine Learning: Science and Technology*, 2021.
- [3] A. Fout, J. Byrd, B. Shariat, and A. Ben-Hur, “Protein interface prediction using graph convolutional networks,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2017.
- [4] L. Ruthotto and E. Haber, “Deep neural networks motivated by partial differential equations,” 2018.
- [5] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” 2016.
- [6] M. Eliasof, E. Haber, and E. Treister, “Pde-gcn: Novel architectures for graph neural networks motivated by partial differential equations,” 2021.
- [7] B. P. Chamberlain, J. Rowbottom, M. Gorinova, S. Webb, E. Rossi, and M. M. Bronstein, “Grand: Graph neural diffusion,” 2021.
- [8] C. Cai and Y. Wang, “A note on over-smoothing for graph neural networks,” 2020.
- [9] S. Luan, C. Hua, Q. Lu, J. Zhu, M. Zhao, S. Zhang, X.-W. Chang, and D. Precup, “Is heterophily a real nightmare for graph neural networks to do node classification?,” 2021.
- [10] C. Bodnar, F. Di Giovanni, B. P. Chamberlain, P. Liò, and M. M. Bronstein, “Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in gnns,” 2022.
- [11] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural ordinary differential equations,” 2019.
- [12] M. Poli, S. Massaroli, J. Park, A. Yamashita, H. Asama, and J. Park, “Graph neural ordinary differential equations,” 2019.
- [13] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.